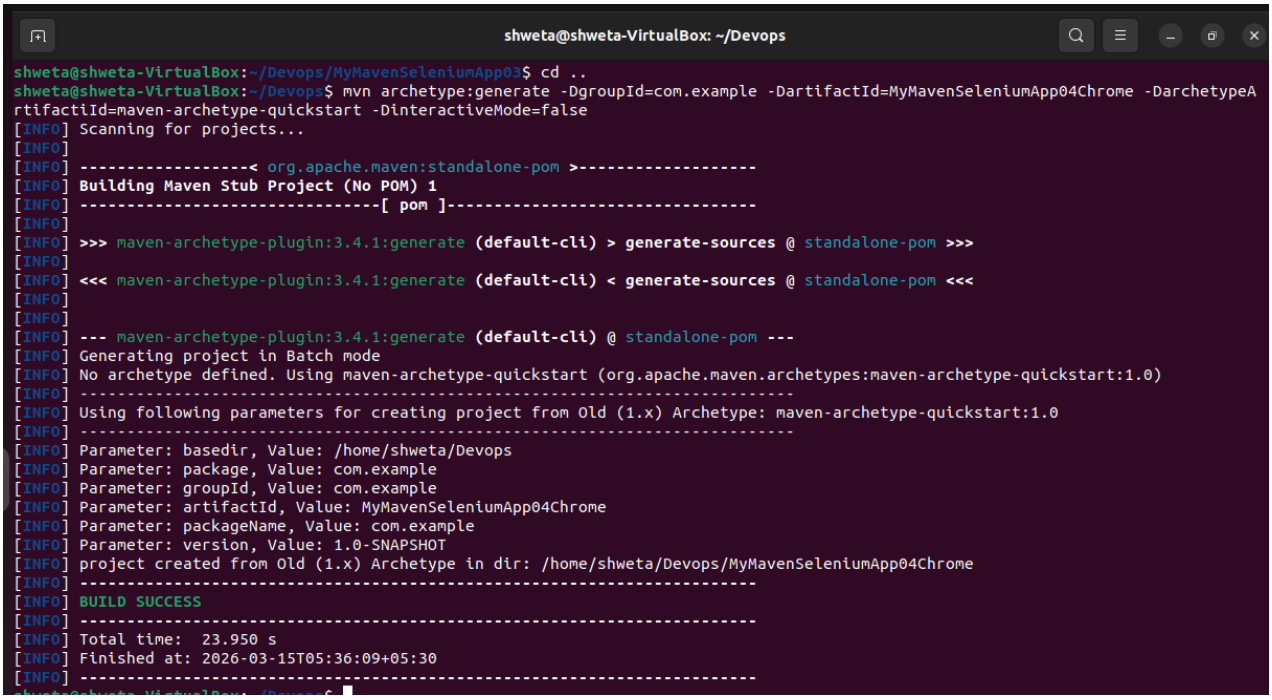
Program 6: Maven Project (all 3 combined) To Connect To Web Servers Using Selenium Dependency

Step 1: create a maven project on terminal

mvn archetype:generate -DgroupId=com.example -DartifactId= MyMavenSeleniumApp02 - DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false



Step 2: Edit the pom.xml file.

Add the Selenium dependency by going to the Maven Central Repository, searching for Selenium, selecting **Selenium Java**, choosing the latest version, and copying the dependency code. Paste this dependency into the pom.xml file.

```
<project
xmlns="http://maven.apache.org/POM/4.0.0
"
xmlns:xsi="http://www.w3.org/2001/XMLS
chema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenSeleniumApp04Chrome</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenSeleniumApp04Chrome</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
```

```
<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
<dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.29.0</version>
</dependency>
  </dependencies>
  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
```

```
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.0.1</version>
       <configuration>
          <archive>
    <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
    </manifestEntries>
          </archive>
        </configuration>
  </plugin> </plugins>
 </build>  </project>
```

3: Before writing the App.java file, inspect the Practice Test Automation webpage to find the element IDs required for automation.

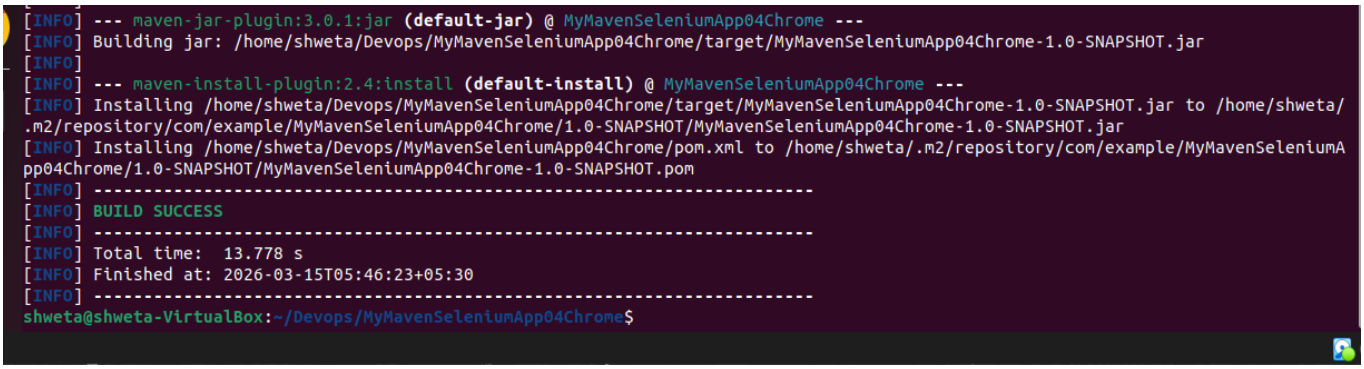
App.java for selenium project

```
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class App {

    public static void main(String[] args) {

        WebDriver driver = new ChromeDriver();

        // ----------- 1. SauceDemo Login -----------
        driver.get("https://www.saucedemo.com/");
        driver.manage().window().maximize();

        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();

        // ----------- 2. Practice Test Login -----------
        driver.get("https://practicetestautomation.com/practice-test-login/");
        driver.manage().window().maximize();

        driver.findElement(By.id("username")).sendKeys("student");
        driver.findElement(By.id("password")).sendKeys("Password123");
        driver.findElement(By.id("submit")).click();
```

```
        // ---------- 3. Automation Exercise Search ----------
        driver.get("https://automationexercise.com/products");
        driver.manage().window().maximize();

        driver.findElement(By.id("search_product")).sendKeys("Blue Top");
        driver.findElement(By.id("submit_search")).click();

        // driver.quit(); // optional
    }
}
```

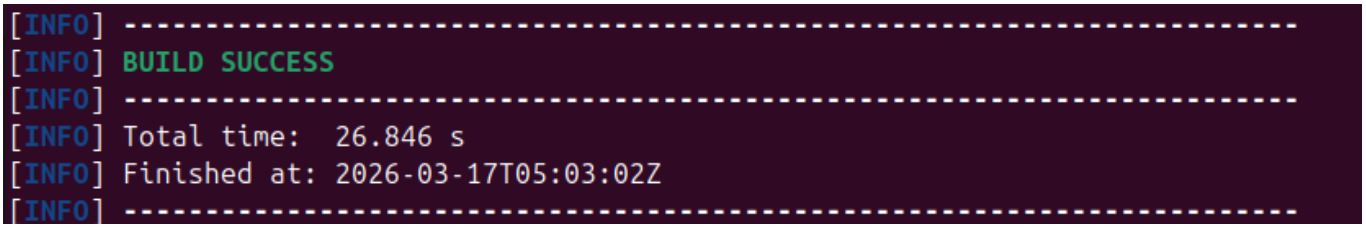
Step 4: build and run the maven application

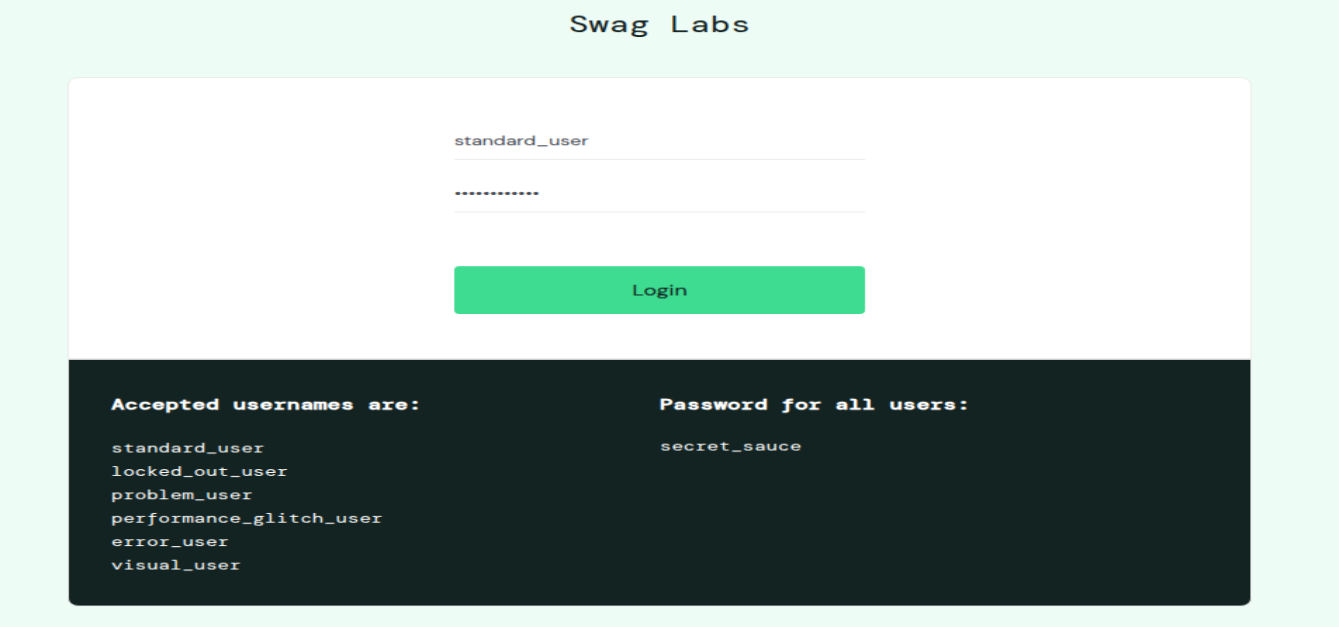
mvn clean install

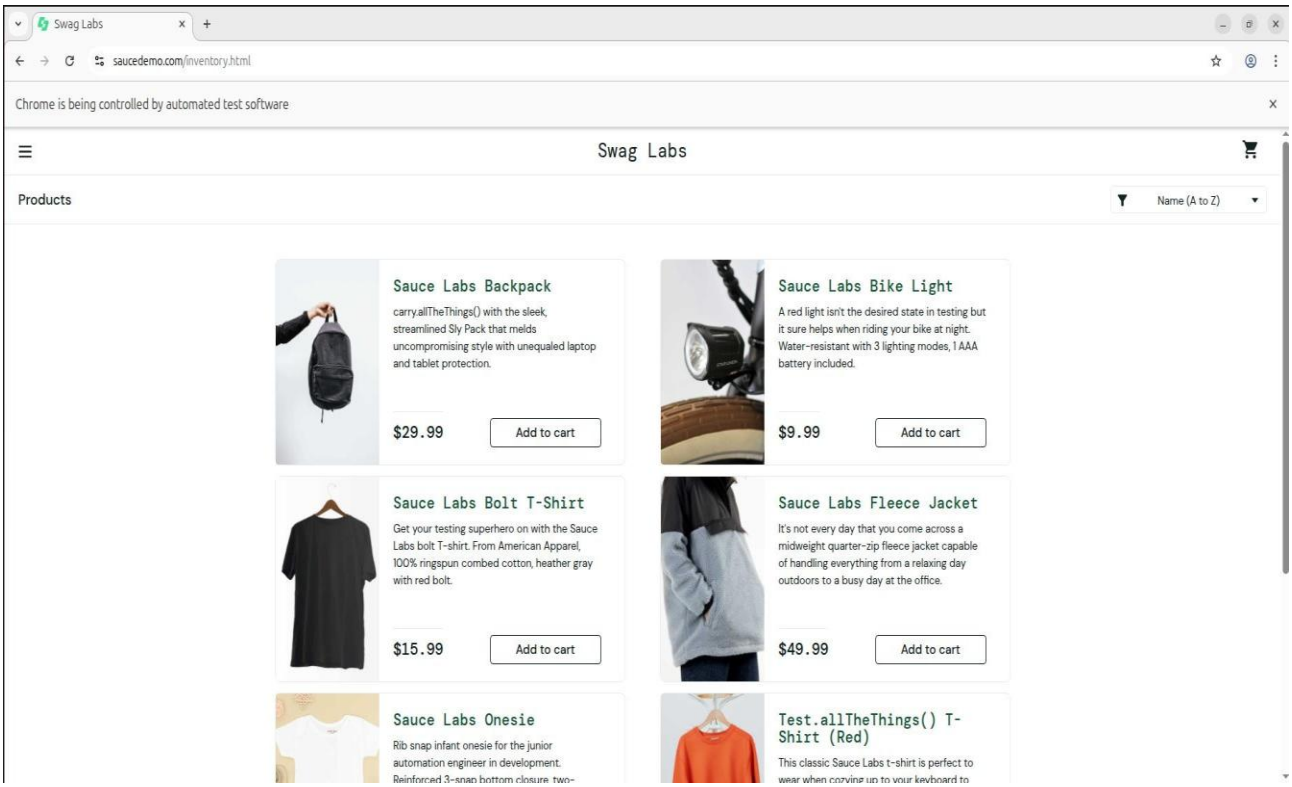


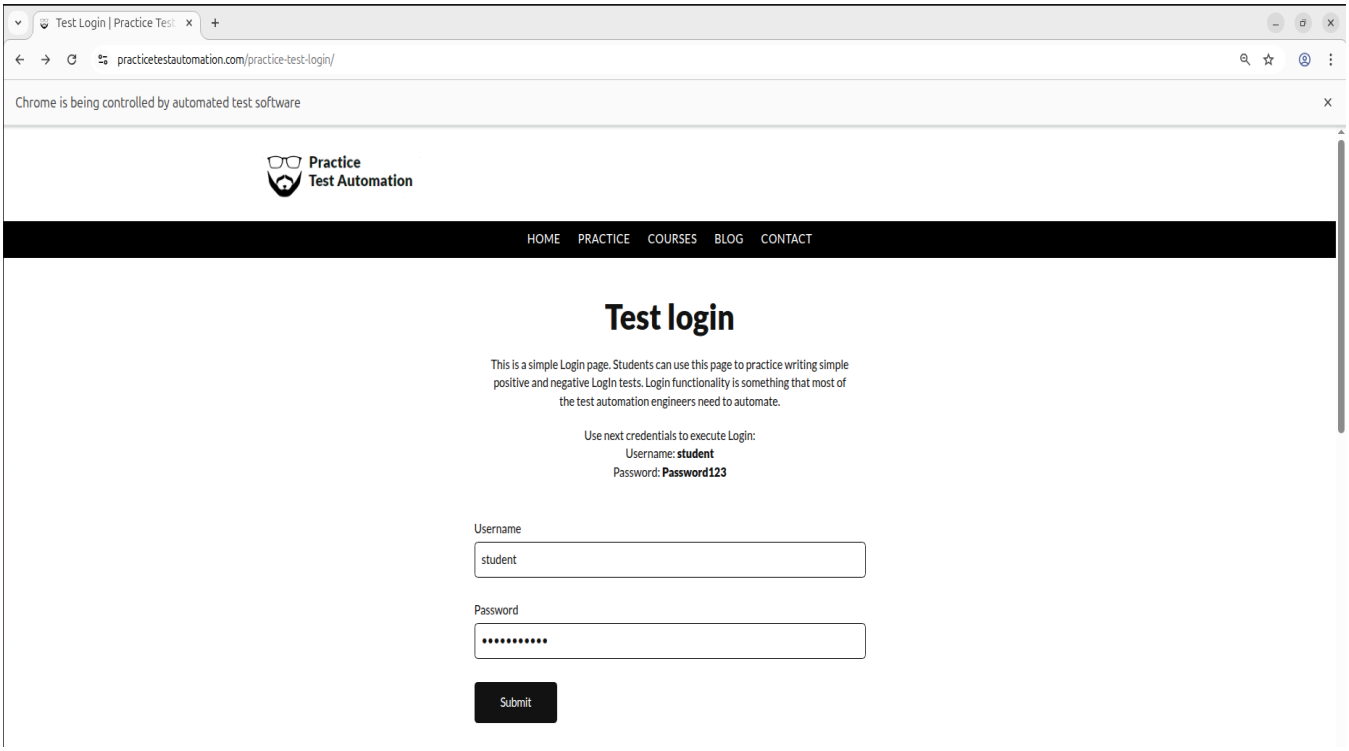
Step 5: run the below command

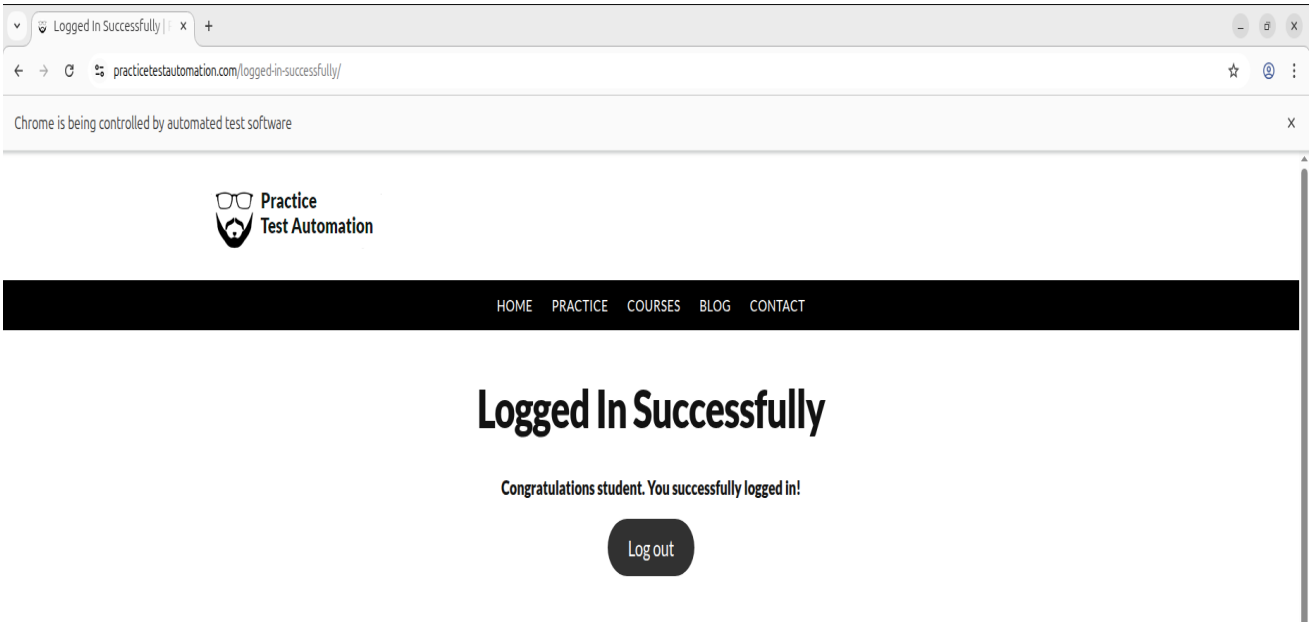
mvn exec:java -Dexec.mainClass="com.example.App"

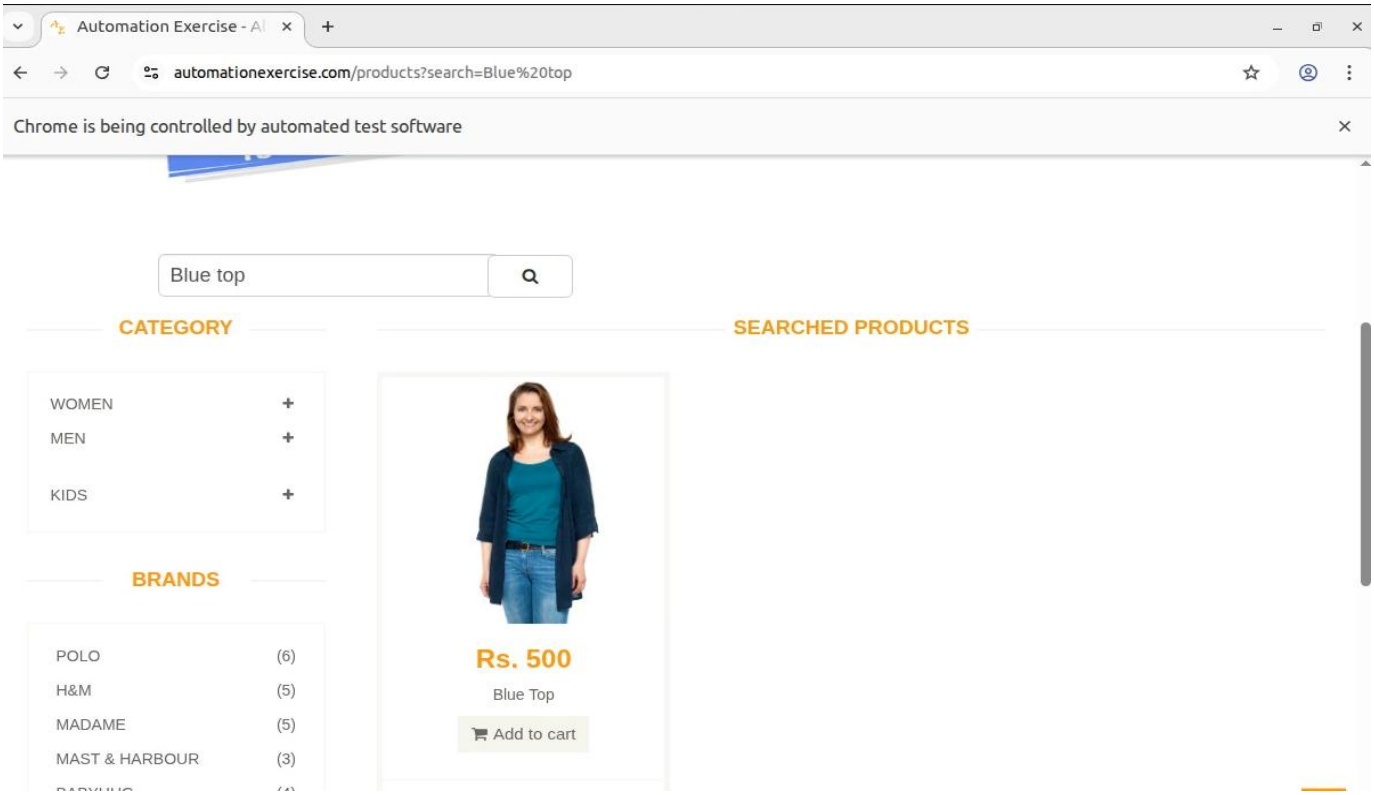



Swag Labs
standard_user
Login
Accepted usernames are:
standard_user
locked_out_user
problem_user
performance_glitch_user
error_user
visual_user
Password for all users:
secret_sauce

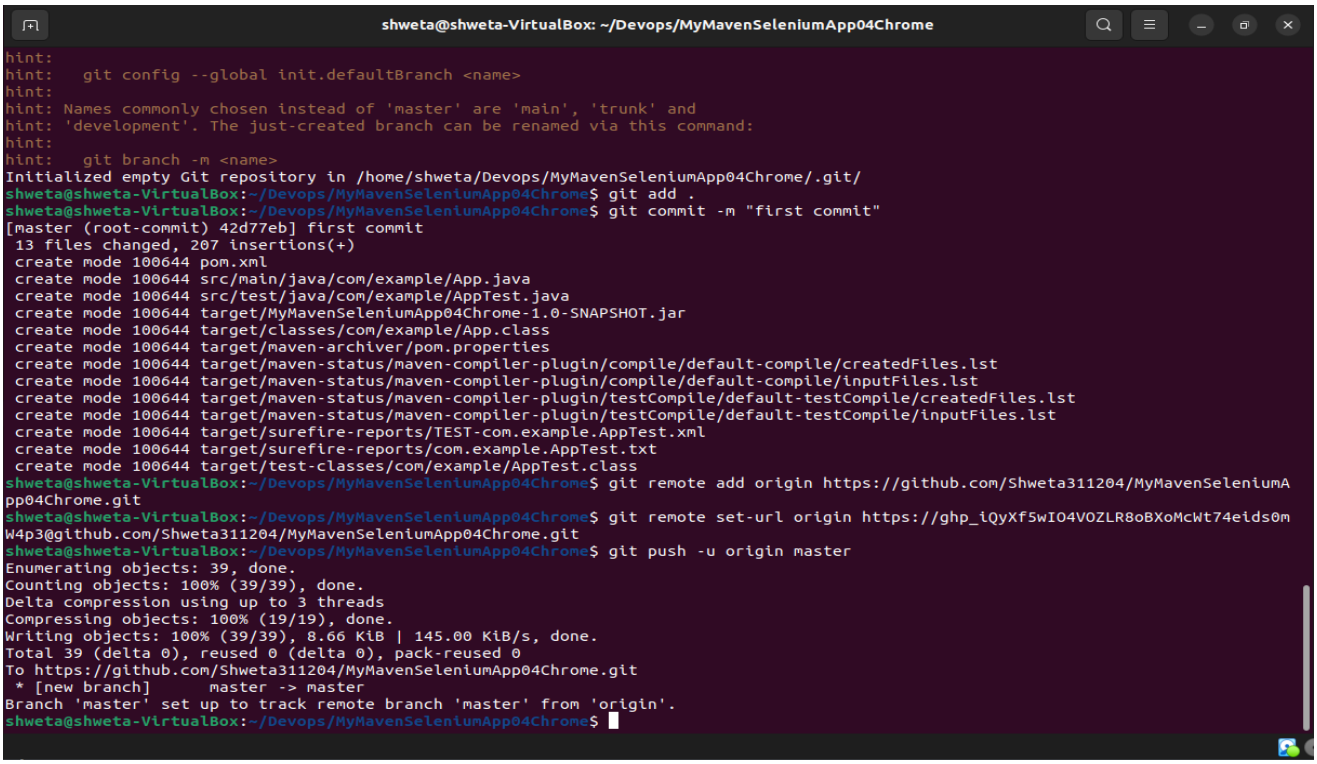

Swag Labs
saucedemo.com/inventory.html
Chrome is being controlled by automated test software
Swag Labs
Products
Name (A to Z)
Sauce Labs Backpack
carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection.
$29.99
Add to cart
Sauce Labs Bike Light
A red light isn't the desired state in testing but it sure helps when riding your bike at night. Water-resistant with 3 lighting modes, 1 AAA battery included.
$9.99
Add to cart
Sauce Labs Bolt T-Shirt
Get your testing superhero on with the Sauce Labs bolt T-shirt. From American Apparel, 100% ringspun combed cotton, heather gray with red bolt.
$15.99
Add to cart
Sauce Labs Fleece Jacket
It's not every day that you come across a midweight quarter-zip fleece jacket capable of handling everything from a relaxing day outdoors to a busy day at the office.
$49.99
Add to cart
Sauce Labs Onesie
Rib snap infant onesie for the junior automation engineer in development. Reinforced 3-snap bottom closure, two-
Test.allTheThings() T-Shirt (Red)
This classic Sauce Labs t-shirt is perfect to wear when cozying up to your keyboard to


Test Login | Practice Test
practicetestautomation.com/practice-test-login/
Chrome is being controlled by automated test software
Practice
Test Automation
HOME PRACTICE COURSES BLOG CONTACT
Test login
This is a simple Login page. Students can use this page to practice writing simple positive and negative LogIn tests. Login functionality is something that most of the test automation engineers need to automate.
Use next credentials to execute Login:
Username: student
Password: Password123
Username
student
Password
Submit


Logged In Successfully
practicetestautomation.com/logged-in-successfully/
Chrome is being controlled by automated test software
Practice
Test Automation
HOME PRACTICE COURSES BLOG CONTACT
Logged In Successfully
Congratulations student. You successfully logged in!
Log out



Pushing MyMavenSeleniumApp04Chrome to Github:



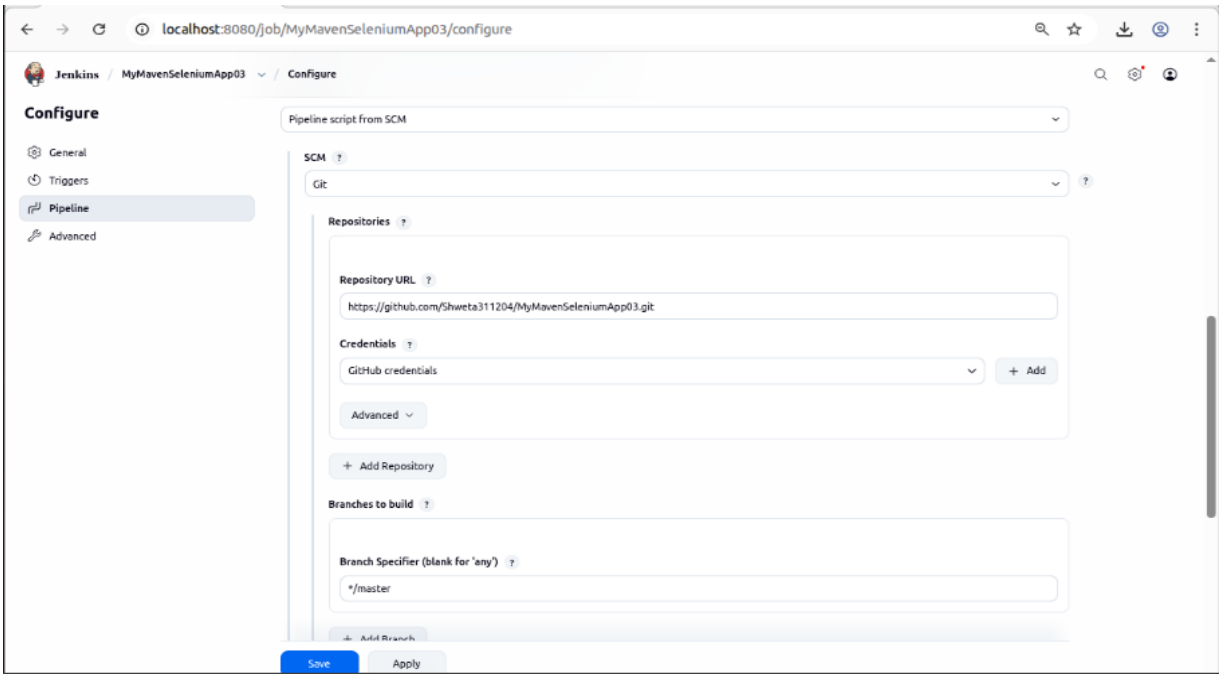
Setting Up Jenkins Pipeline for MyMavenSeleniumApp04Chrome

Step 1: To deploy **MyMavenSeleniumApp04Chrome,** open the terminal, cd into the **MyMavenSeleniumApp04Chrome** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

```
pipeline {

agent any  // Use any available agent tools {

maven 'Maven'  // Ensure this matches the name configured in Jenkins
}

stages {

stage('Checkout') { steps {

git branch: 'master', url:
'https://github.com/Shweta311204/MyMavenSeleniumApp04Chrome.git'
}
}

stage('Build') { steps {

sh 'mvn clean package'  // Run Maven build
}
}

stage('Test') { steps {

sh 'mvn test'  // Run unit tests
}
}

stage('Run Application') { steps {

// Start the JAR application
```

```
sh 'mvn exec:java -Dexec.mainClass="com.example.App"'
}
}
}

post {

success {

echo 'Build and deployment successful!'
}

failure {

echo 'Build failed!'
}
}
}
```

Step 2: To deploy **MyMavenSeleniumApp04**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on **"New Item"** to create a job, enter the item name (e.g., *MyMavenSeleniumApp04*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click **"OK"** to proceed to the job configuration page. In **MyMavenSeleniumApp04**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenSeleniumApp04**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MyMavenSeleniumApp04 folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.



Step 4: After clicking **Apply and Save**, go back to **MyMavenSeleniumApp04 in Jenkins** and click
Build Now. If you get an error like:

java.lang.NoClassDefFoundError: io/github/bonigarcia/wdm/WebDriverManager

this error occurs when the application is unable to find required external libraries (such as Selenium WebDriver and WebDriverManager) at runtime. Even though the project builds successfully, the dependencies are not included inside the final JAR file, so Jenkins fails while executing the program.

To solve this issue, instead of running the application using a JAR file, we use Maven execution so that all dependencies defined in pom.xml are properly loaded during runtime execution.

This ensures that Selenium and WebDriverManager libraries are available when the program runs in Jenkins.

Replace the existing execution method with the following:

mvn exec:java -Dexec.mainClass="com.example.App"

Jenkins Pipeline Update:

```
stage('Run Application') { steps {

sh 'mvn exec:java -Dexec.mainClass="com.example.App"'
}
}
```

This stage is used to execute the Java application directly through Maven so that all dependencies defined in the project are included during runtime.

pom.xml Dependencies (Required Fix):

```
<dependency>

<groupId>org.seleniumhq.selenium</groupId>

<artifactId>selenium-java</artifactId>

<version>4.29.0</version>

</dependency>

<dependency>

<groupId>io.github.bonigarcia</groupId>

<artifactId>webdrivermanager</artifactId>

<version>5.7.0</version>

</dependency>
```

These dependencies are required to enable Selenium automation and automatic browser driver management during execution.

Changes made in App.java to solve the error:

To resolve the **NoClassDefFoundError (WebDriverManager missing at runtime)**, the following changes were ensured in App.java:

Added **WebDriverManager setup** to automatically handle ChromeDriver dependency: WebDriverManager.chromedriver().setup();

This line automatically downloads, sets up, and manages the correct version of ChromeDriver required for the installed Chrome browser. It removes the need to manually download or configure the driver path and ensures compatibility between Chrome and ChromeDriver during Jenkins execution.

Configured **ChromeOptions for Jenkins/Linux environment** to avoid browser crash in headless mode:

ChromeOptions options = new ChromeOptions(); options.addArguments("--headless=new"); options.addArguments("--no-sandbox"); options.addArguments("--disable-dev-shm-usage"); options.addArguments("--disable-gpu"); options.addArguments("--remote-allow-origins=*");

These options ensure that Chrome runs in a headless (no UI) mode suitable for Jenkins/Linux servers and prevent common issues like browser crash, sandbox errors, and memory limitations during execution.

Used **ChromeDriver with options** instead of normal driver: WebDriver driver = new ChromeDriver(options);

This ensures that the browser is launched with the configured ChromeOptions settings (such as headless mode and Jenkins-safe arguments), making the automation stable and suitable for execution in CI/CD environments like Jenkins.

Ensured proper **login automation flow** after driver initialization:

driver.get("https://www.automationexercise.com/"); driver.findElement(By.id("user-name")).sendKeys("standard_user");
driver.findElement(By.id("password")).sendKeys("secret_sauce");
driver.findElement(By.id("login-button")).click();

This performs the complete login automation flow in the automation exercise application by opening the website, entering valid credentials, and clicking the login button to validate successful user authentication.

The following are the updated pom.xml, App.java, and Jenkinsfile: pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"

        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0

        http://maven.apache.org/maven-v4_0_0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>

    <artifactId>MyMavenSeleniumApp03</artifactId>

    <version>1.0-SNAPSHOT</version>

    <packaging>jar</packaging>

    <name>MyMavenSeleniumApp03</name>

    <!-- Java Version -->

    <properties>

       <maven.compiler.source>11</maven.compiler.source>
```

```
    <maven.compiler.target>11</maven.compiler.target>
</properties>
<dependencies>
    <!-- JUnit (Test only) -->
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
    <!-- Selenium -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.41.0</version>
    </dependency>
</dependencies>
<build>
    <plugins>
```

```
<!-- Compiler Plugin -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.11.0</version>
</plugin>
<!-- Surefire Plugin (for tests) -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.2.5</version>
</plugin>
<!-- Shade Plugin (creates fat JAR with dependencies) -->
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-shade-plugin</artifactId>
    <version>3.5.0</version>
    <executions>
        <execution>
            <phase>package</phase>
```

```
                <goals>
                    <goal>shade</goal>
                </goals>
                <configuration>

<createDependencyReducedPom>false</createDependencyReducedPom>
                    <transformers>
                        <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTran
sformer">
                            <mainClass>com.example.App</mainClass>
                        </transformer>
                    </transformers>
                </configuration>
            </execution>
        </executions>
    </plugin>
  </plugins>
 </build>
</project>
```

App.java

```
package com.example;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.chrome.ChromeOptions;

public class App {

    public static void main(String[] args) {

        //  Create options FIRST

        ChromeOptions options = new ChromeOptions();

        options.addArguments("--headless=new");

        options.addArguments("--no-sandbox");

        options.addArguments("--disable-dev-shm-usage");

        options.addArguments("--remote-allow-origins=*");

        // Pass options to driver

        WebDriver driver = new ChromeDriver(options);

        driver.get("http://practicetestautomation.com/practice-test-login/");

        driver.findElement(By.id("username")).sendKeys("student");

        driver.findElement(By.id("password")).sendKeys("Password123");

        driver.findElement(By.id("submit")).click();
```

```
        System.out.println("Login successful, Title: " + driver.getTitle());
        driver.quit();
    }
}
```

Jenkinsfile:

```
pipeline {
    agent any  // Use any available agent
   tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url:
'https://github.com/Shweta311204/MyMavenSeleniumApp04Chrome.git'
            }
        }
        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
```

```
        }
    }
    stage('Test') {
        steps {
            sh 'mvn test'  // Run unit tests
        }}
            stage('Run Application') {
        steps {
            // Start the JAR application
            sh 'java -jar target/MyMavenSeleniumApp02-1.0-SNAPSHOT.jar'
        } } }
post {
    success {
        echo 'Build and deployment successful!' }
            failure {
        echo 'Build failed!'
    }}}
```

Step 5: After making the changes, go back to **MyMavenSeleniumApp04** in Jenkins and click **Build Now**. The build should now complete successfully without any errors.

This happens because the dependencies (such as **Selenium and WebDriverManager**) are properly included in the project through Maven configuration, and the application is executed using mvn exec:java. So during runtime, Jenkins is able to load all required libraries correctly, and issues like **java.lang.NoClassDefFoundError** are resolved.